

Plumber

- Creates a Web API by decorating existing R code with special comments

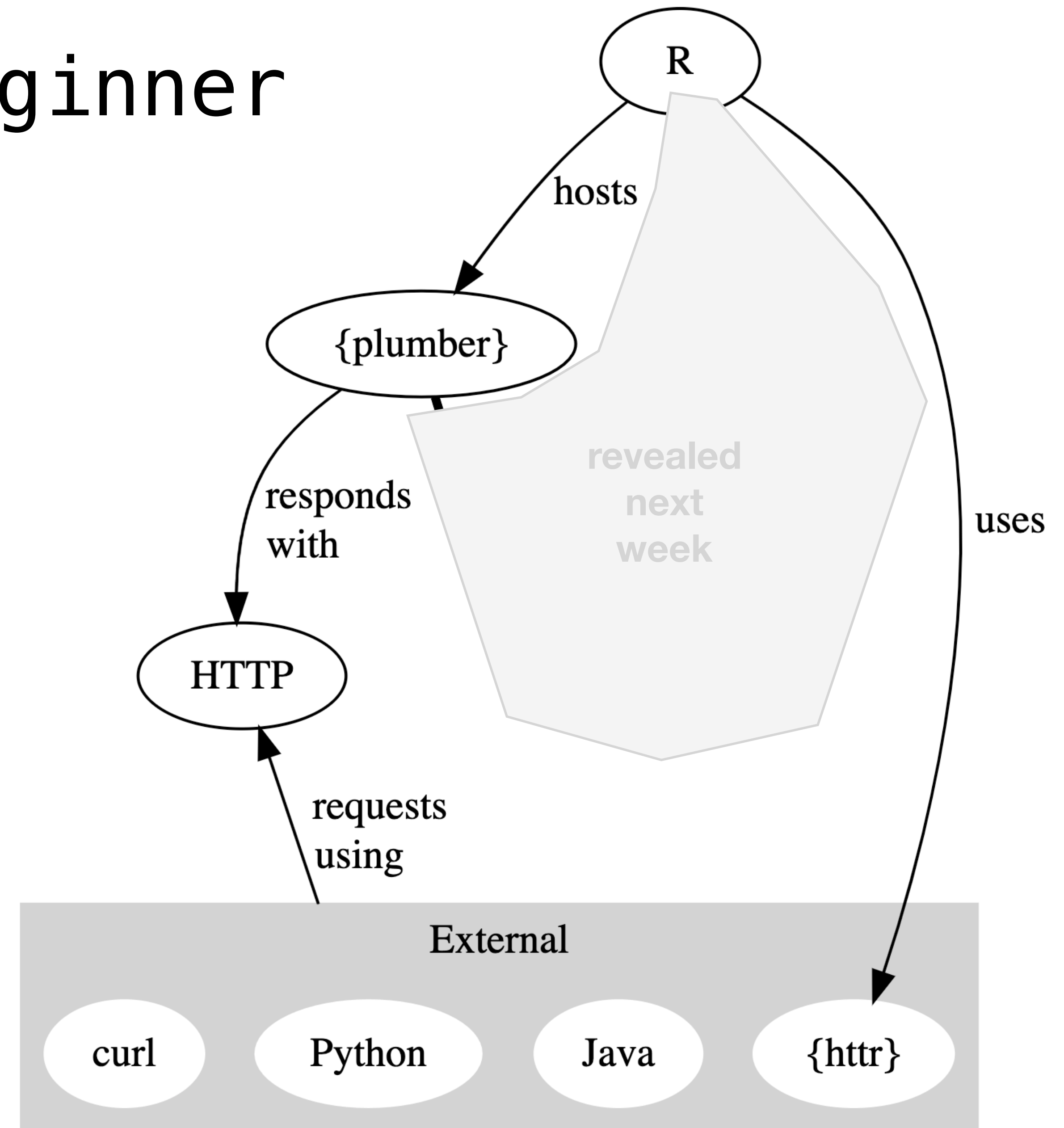
Web APIs

- Use HTTP to communicate information to other computers
- Each Url “route” defines a computation path
- A client sends a request, a server sends a response
- Web browsers make many Web API calls to every website you visit

```
#' @apiTitle Plumber Beginner
```

```
## @get /ask
```

```
function(question) {  
  barret::answer(  
    question  
  )  
}
```



```
# ./plumber.R
```

```
## @get /fast/<id>  
function(id) {  
  fast_calc(id)  
}
```

```
## @get /slow/<id>  
function(id) {  
  slow_calc(id)  
}
```





Receive



Process

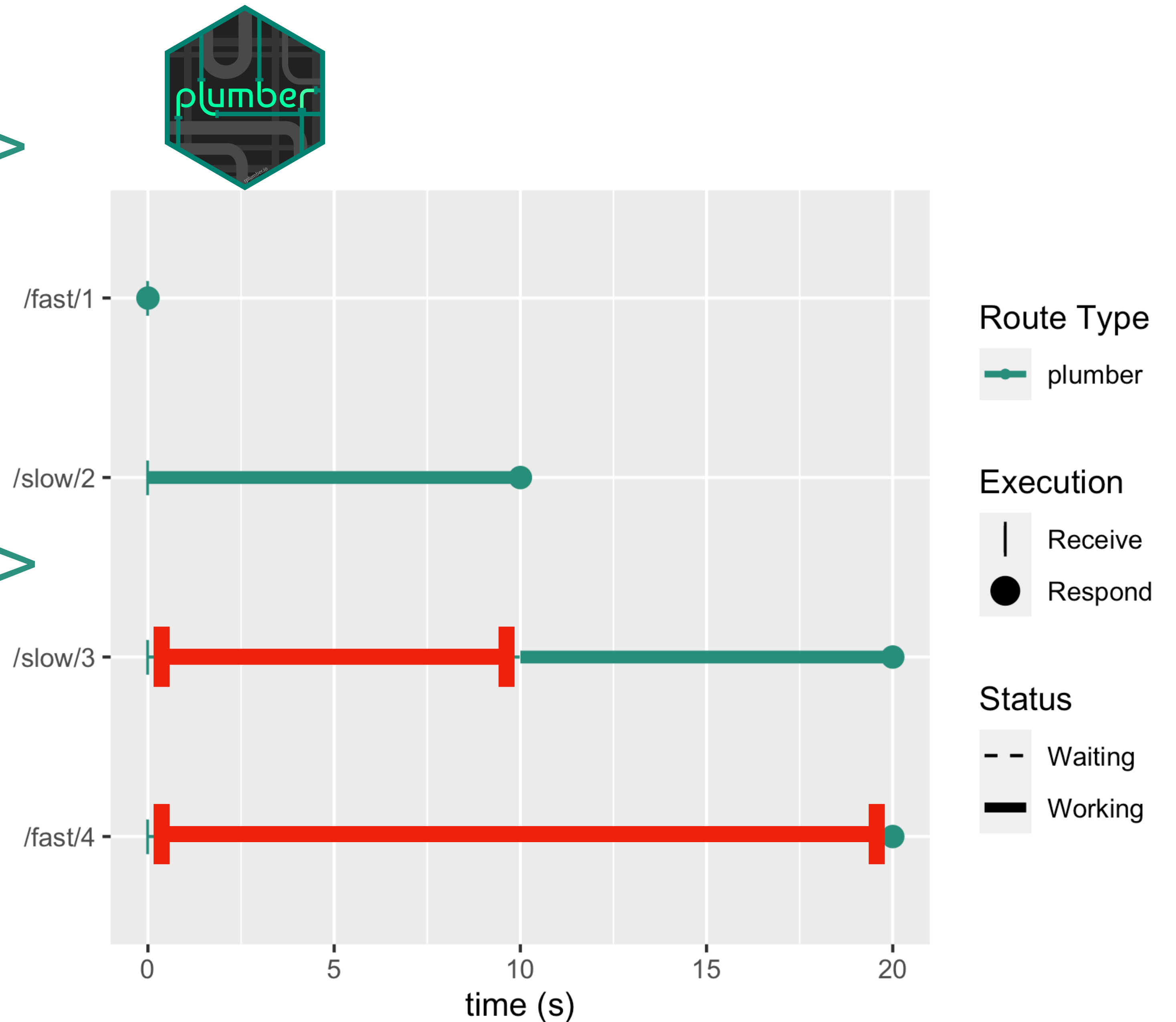


Respond



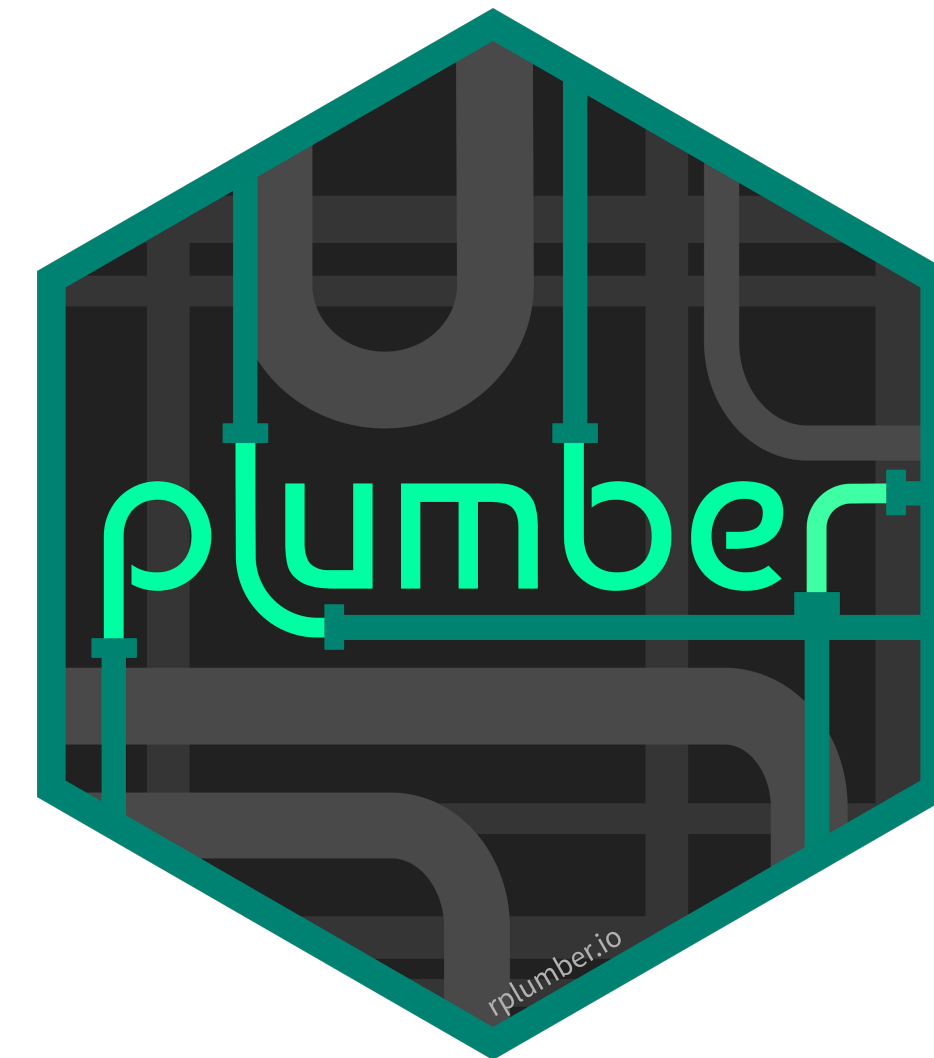
```
## @get /fast/<id>
function(id) {
  fast_calc(id)
}
```

```
## @get /slow/<id>
function(id) {
  slow_calc(id)
}
```



```
## @get /fast/<id>  
function(id) {  
  fast_calc(id)  
}
```

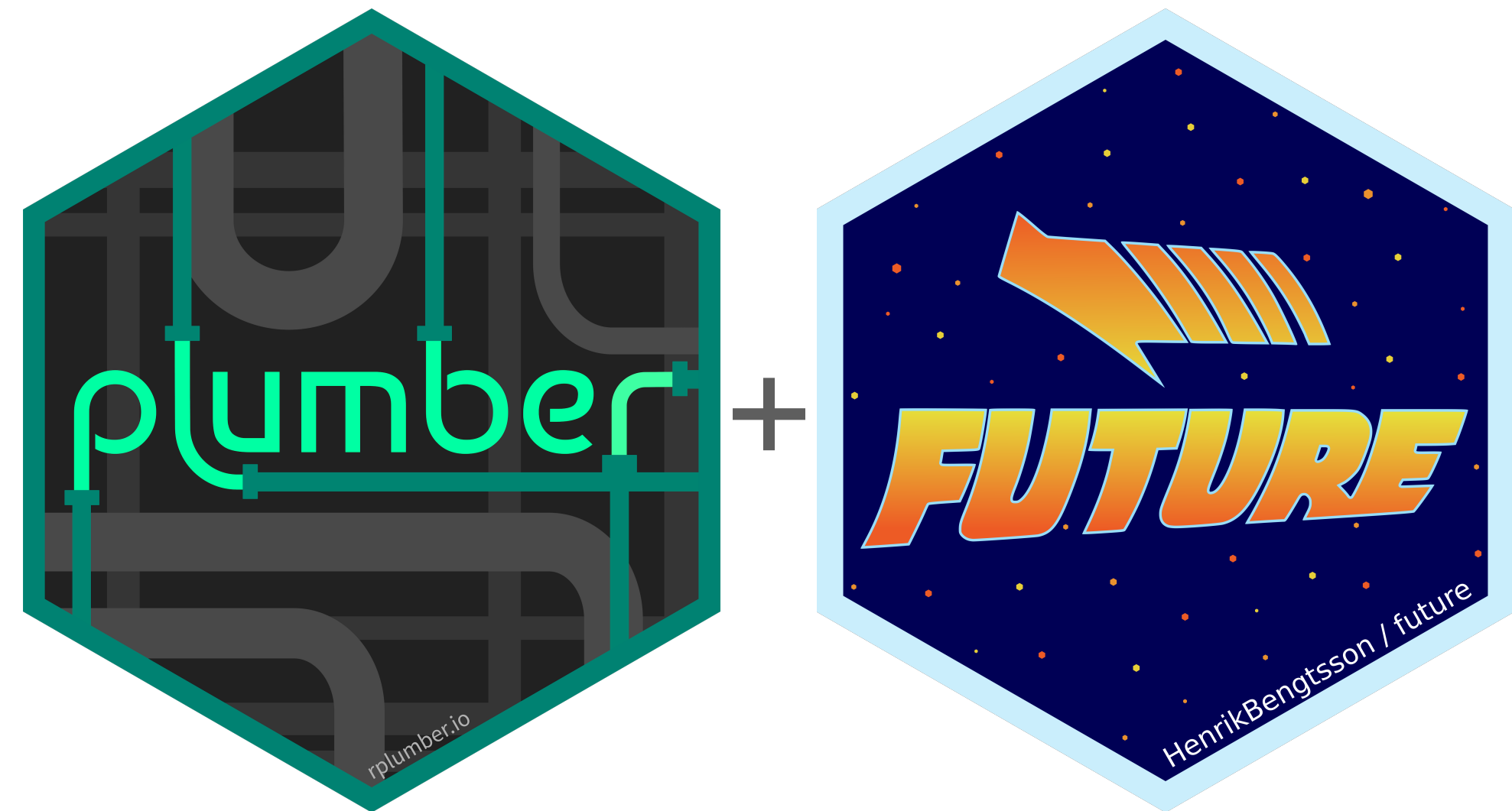
```
## @get /slow/<id>  
function(id) {  
  slow_calc(id)  
}
```



```
future::plan("multisession")
```

```
## @get /fast/<id>  
function(id) {  
  fast_calc(id)  
}
```

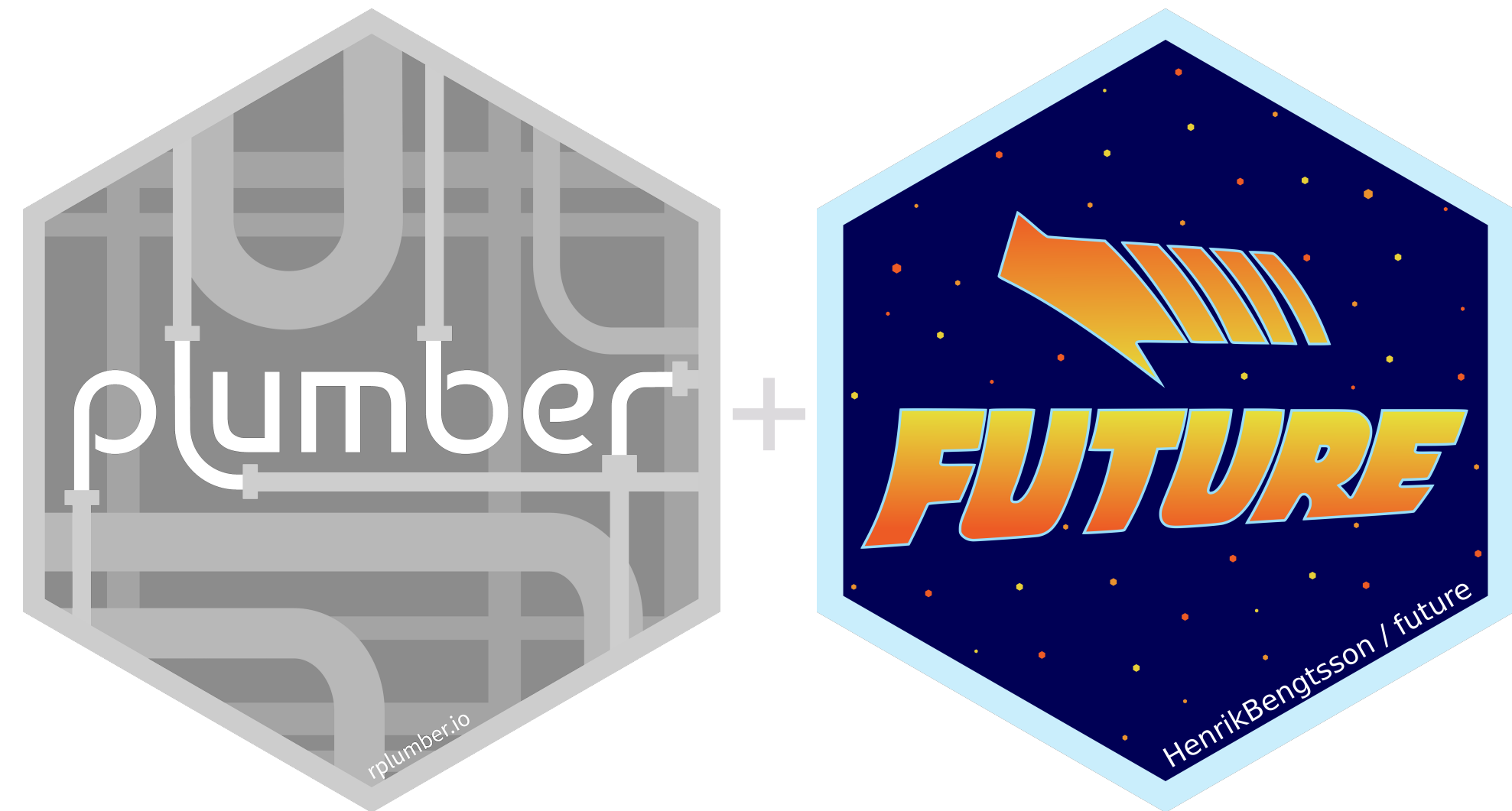
```
## @get /slow/<id>  
function(id) {  
  future::future({  
    slow_calc(id)  
  })  
}
```



```
future::plan("multisession")
```

```
## @get /fast/<id>  
function(id) {  
  fast_calc(id)  
}
```

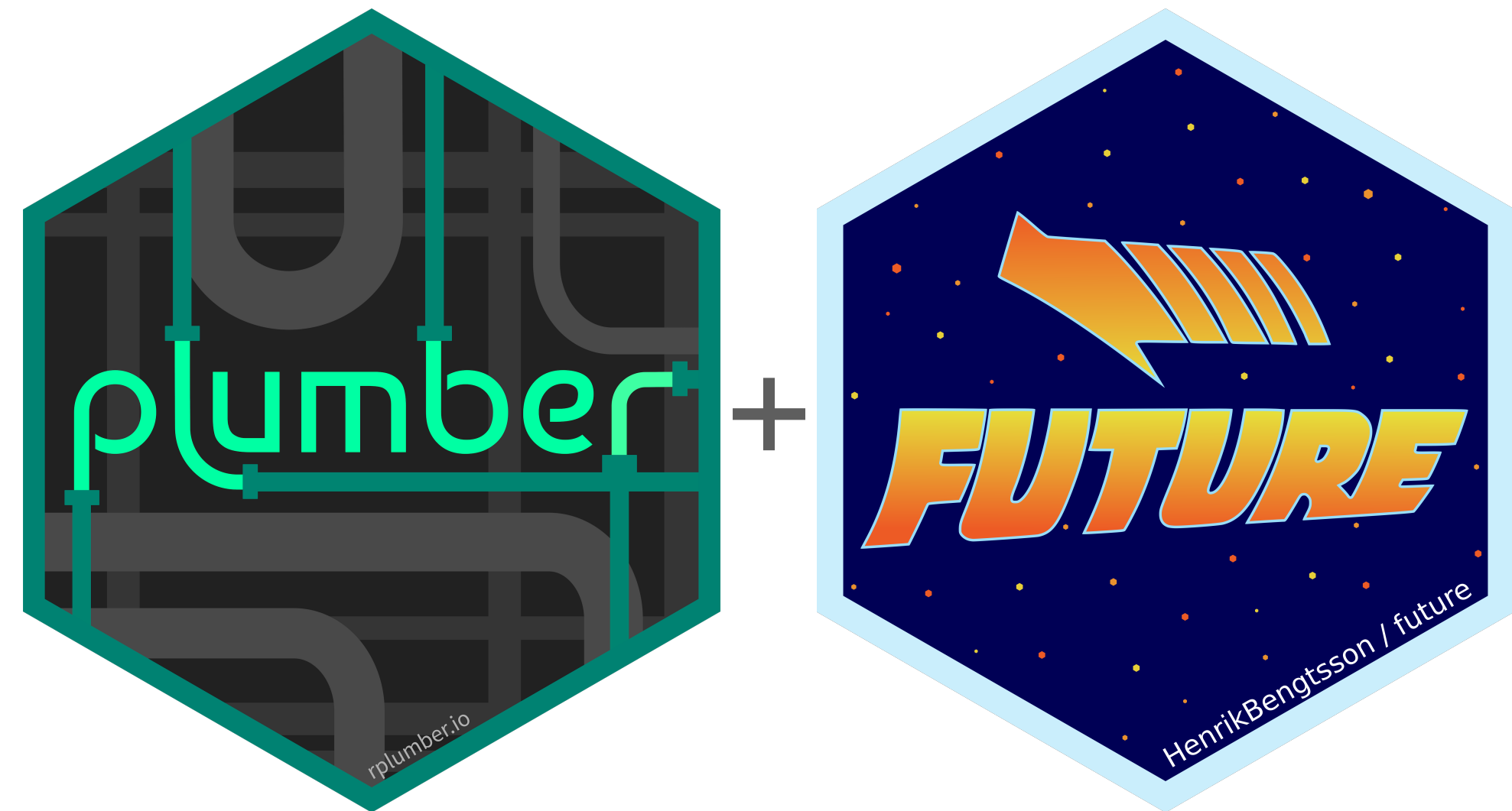
```
## @get /slow/<id>  
function(id) {  
  future::future({  
    slow_calc(id)  
  })  
}
```



```
future::plan("multisession")
```

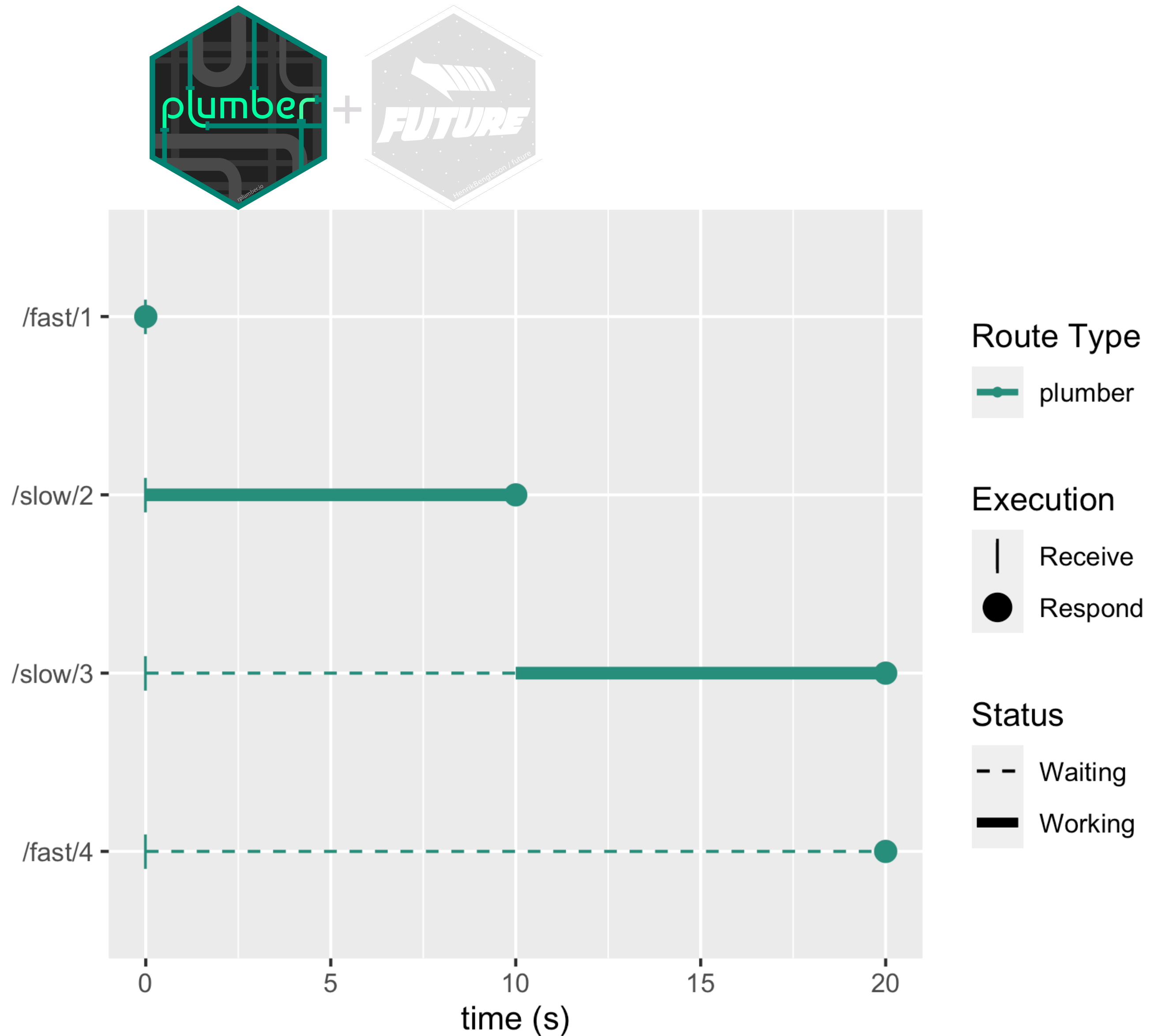
```
## @get /fast/<id>  
function(id) {  
  fast_calc(id)  
}
```

```
## @get /slow/<id>  
function(id) {  
  future::future({  
    slow_calc(id)  
  })  
}
```



```
## @get /fast/<id>
function(id) {
  fast_calc(id)
}
```

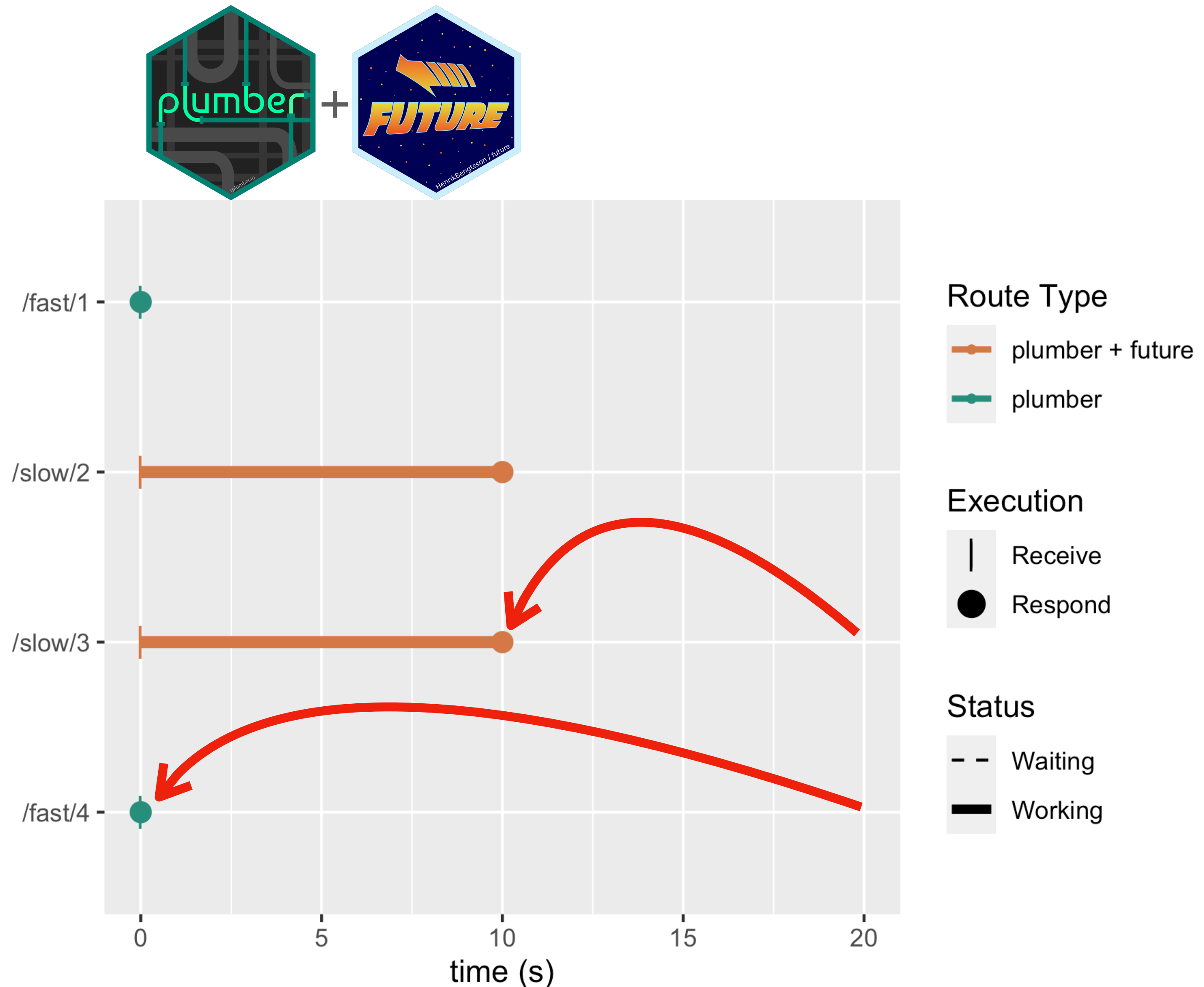
```
## @get /slow/<id>
function(id) {
  slow_calc(id)
}
```

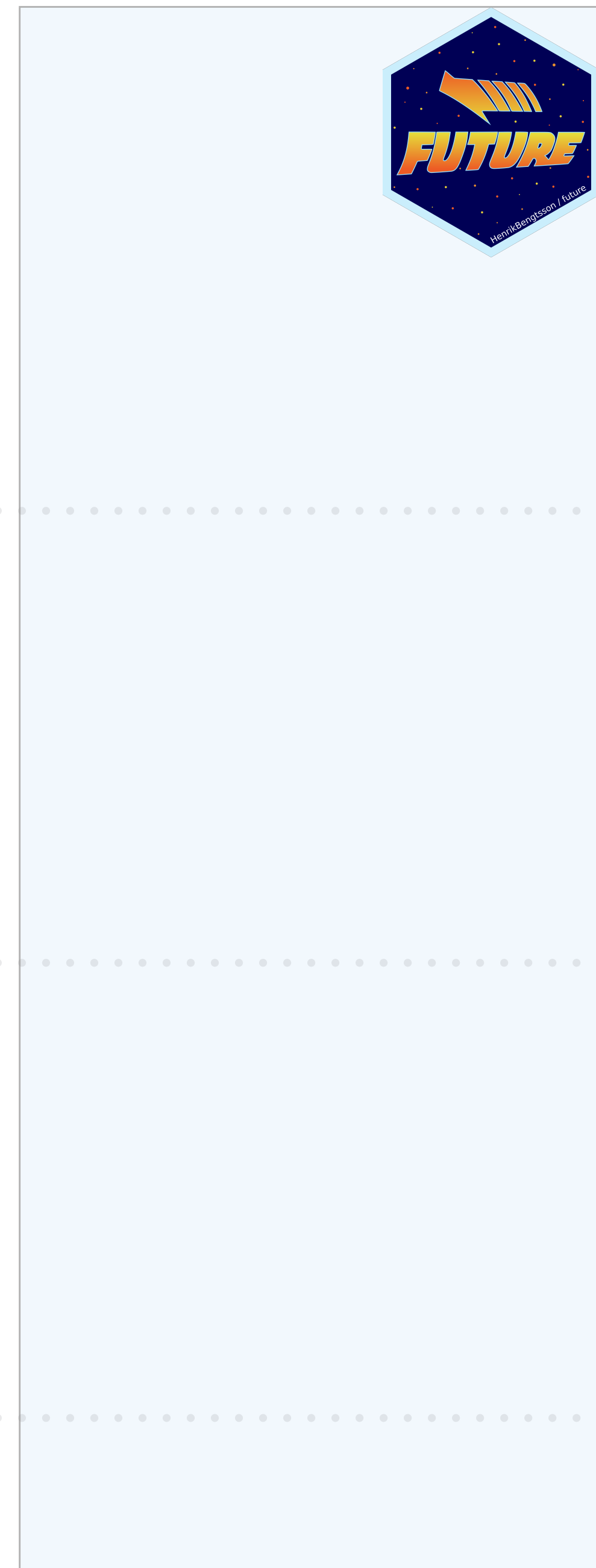
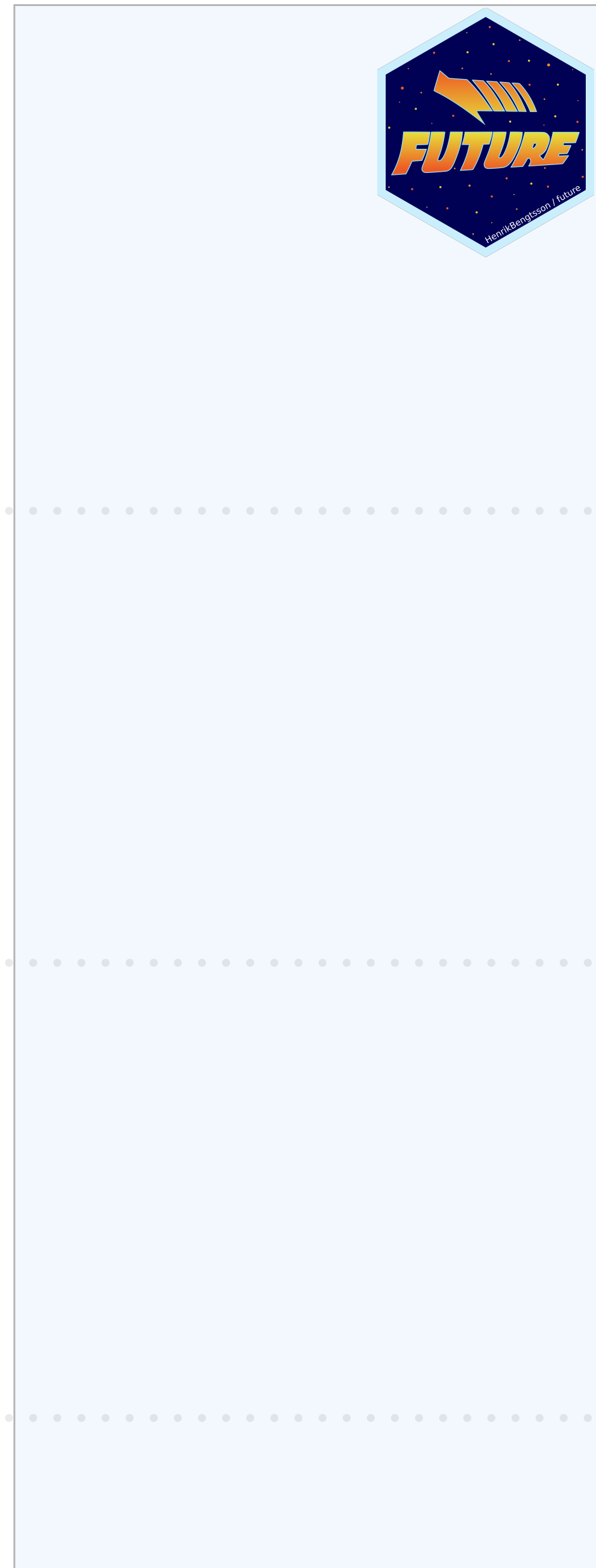


```
future::plan("multisession")
```

```
## @get /fast/<id>  
function(id) {  
  fast_calc(id)  
}
```

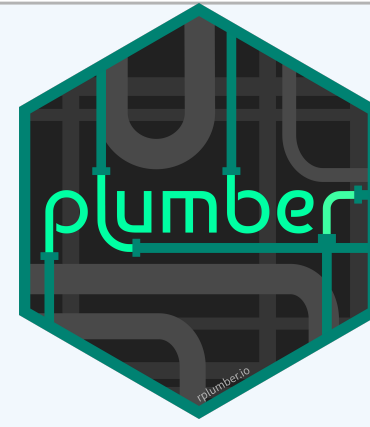
```
## @get /slow/<id>  
function(id) {  
  future::future({  
    slow_calc(id)  
  })  
}
```







Main



Worker
#1



Worker
#2



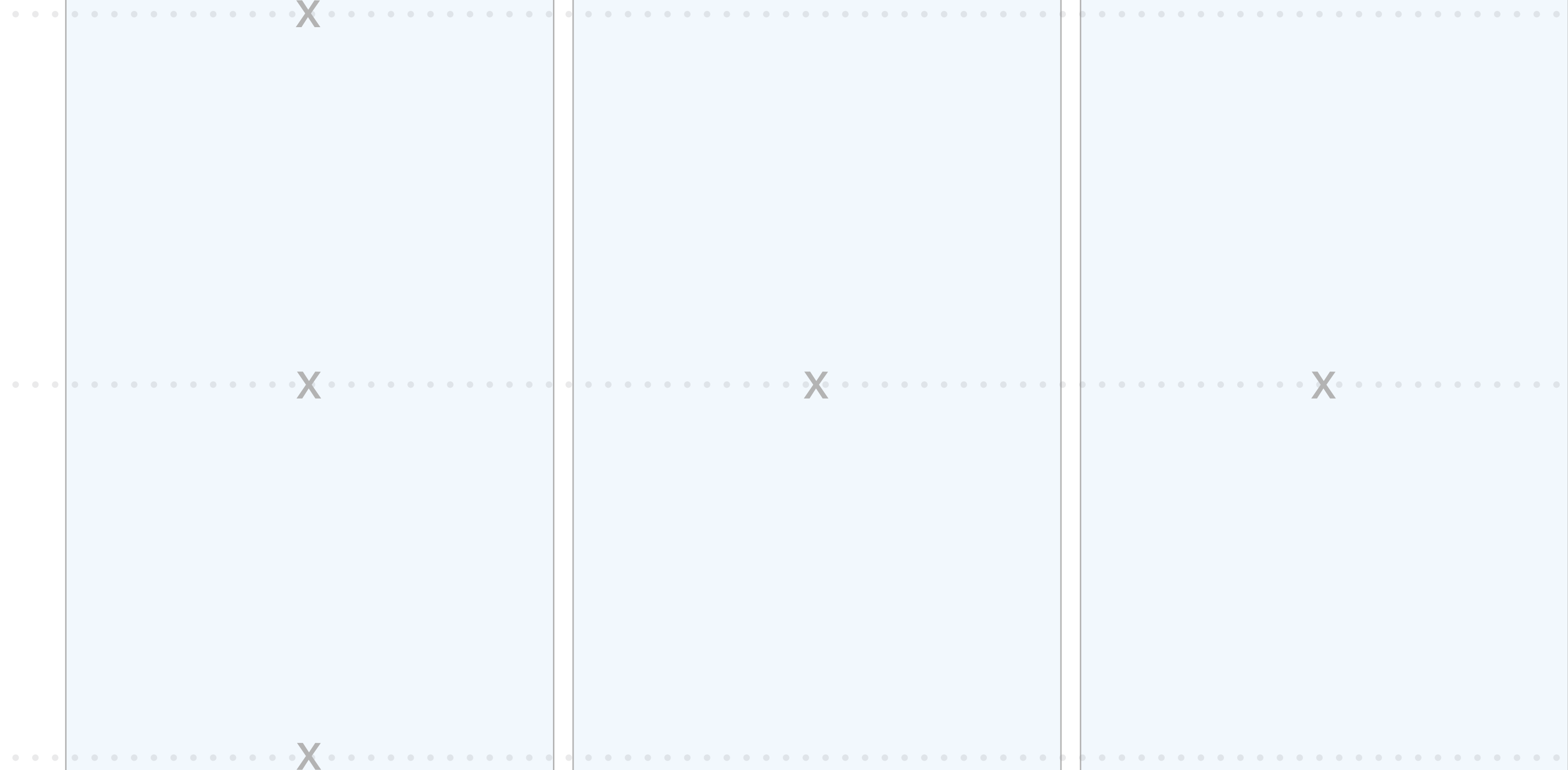
Receive



Process



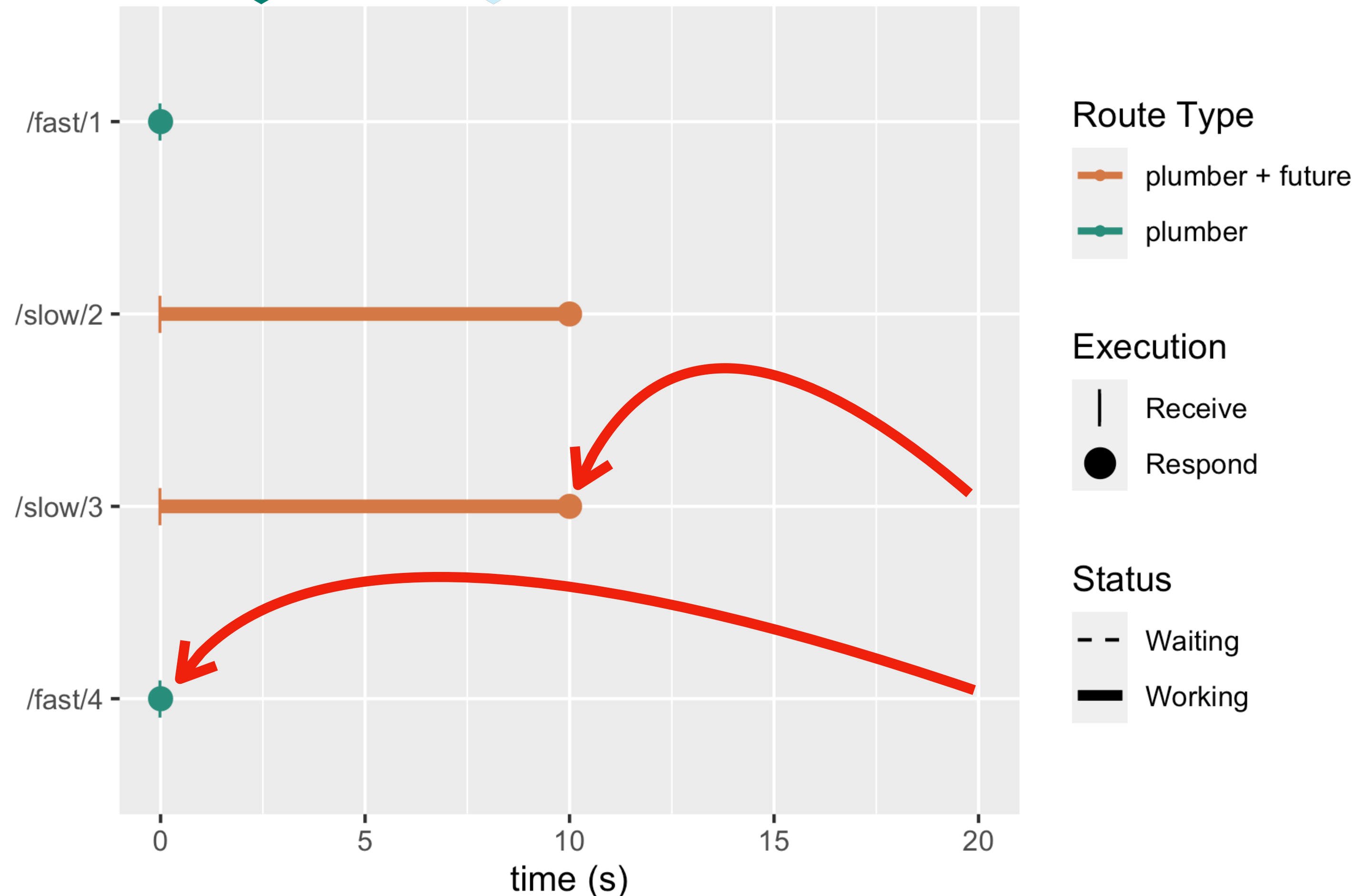
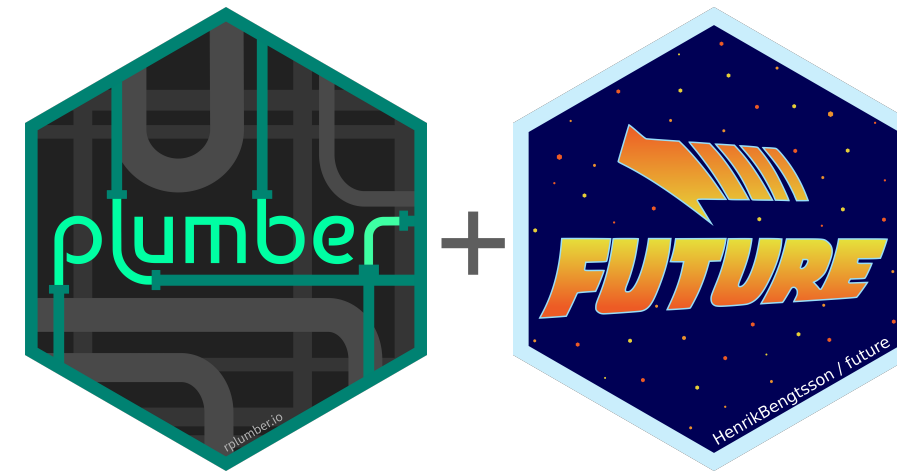
Respond



```
future::plan("multisession")
```

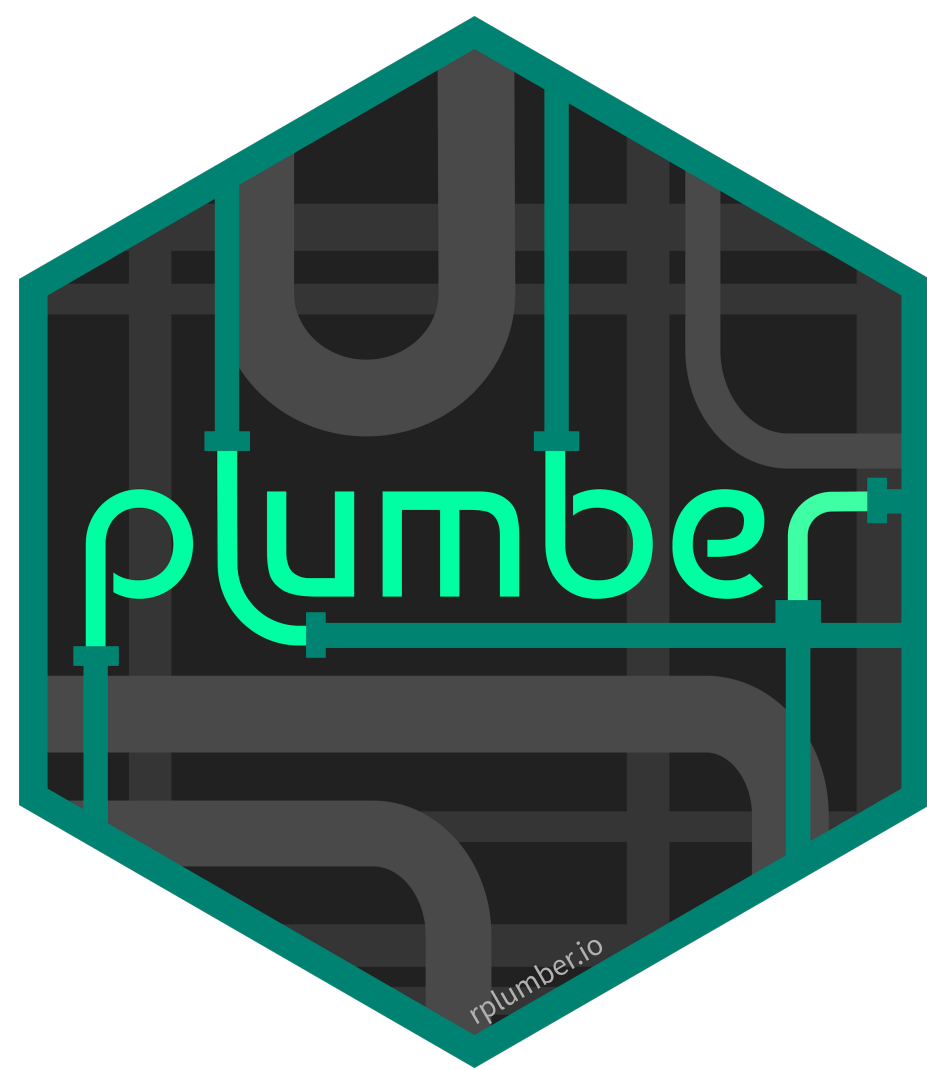
```
## @get /fast/<id>  
function(id) {  
  fast_calc(id)  
}
```

```
## @get /slow/<id>  
function(id) {  
  future::future({  
    slow_calc(id)  
  })  
}
```



Limitations...

- Manually add `future(expr)`
- `future(expr)` is not instant
- Processing power is finite
Memory size is finite



+



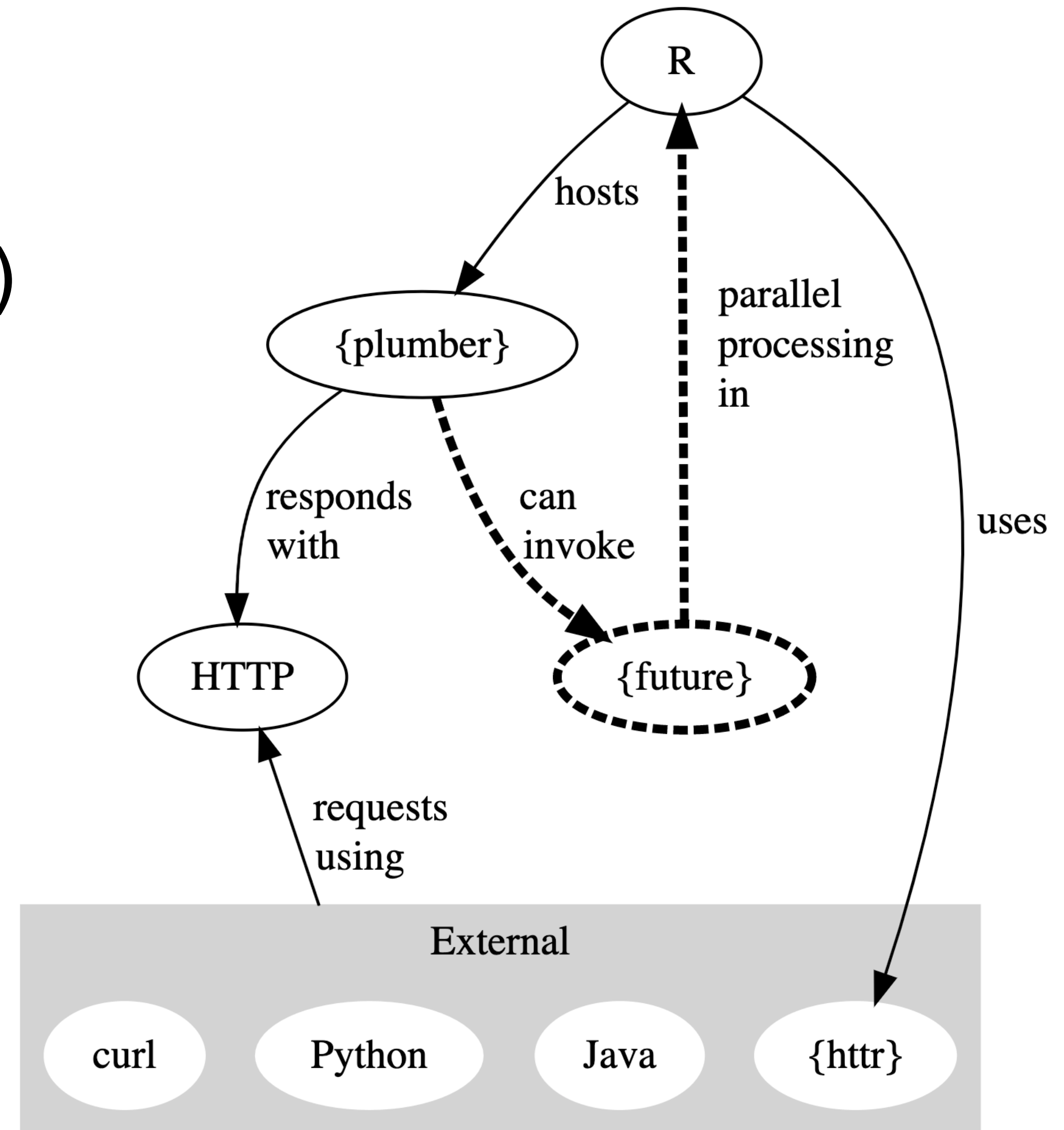
Barret Schloerke
RStudio / Shiny Team
  @schloerke

- Offload *slow* routes using `future(expr)`
- **Slides and material:**
rstudio.io/global2021/barretschloerke

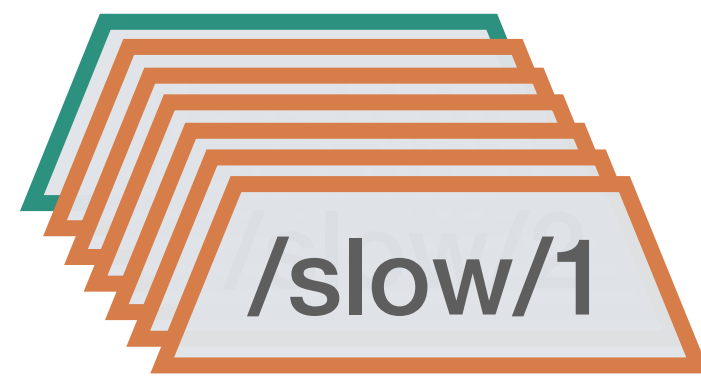
```

#* @get /ask
function(question) {
  if (is_quick(question))
    barret::answer(
      question)
  else
    future({
      barret::answer(
        question)
    })
}

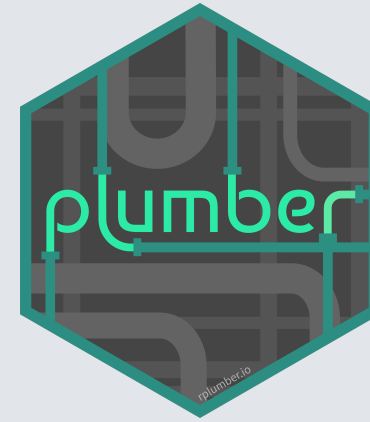
```



Question:
What happens
when I run out of
future workers?



Main



Blocked!!

Worker
#1



Worker
#2



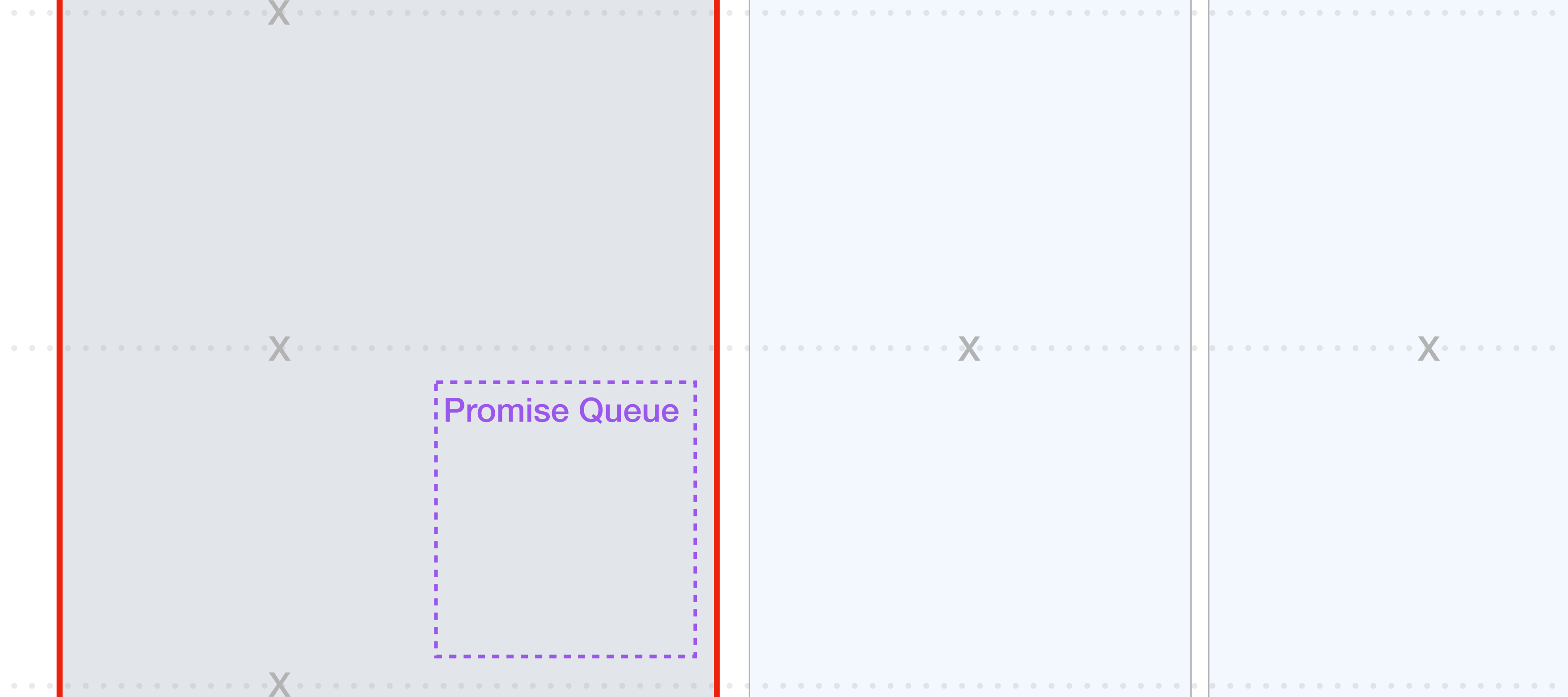
Receive



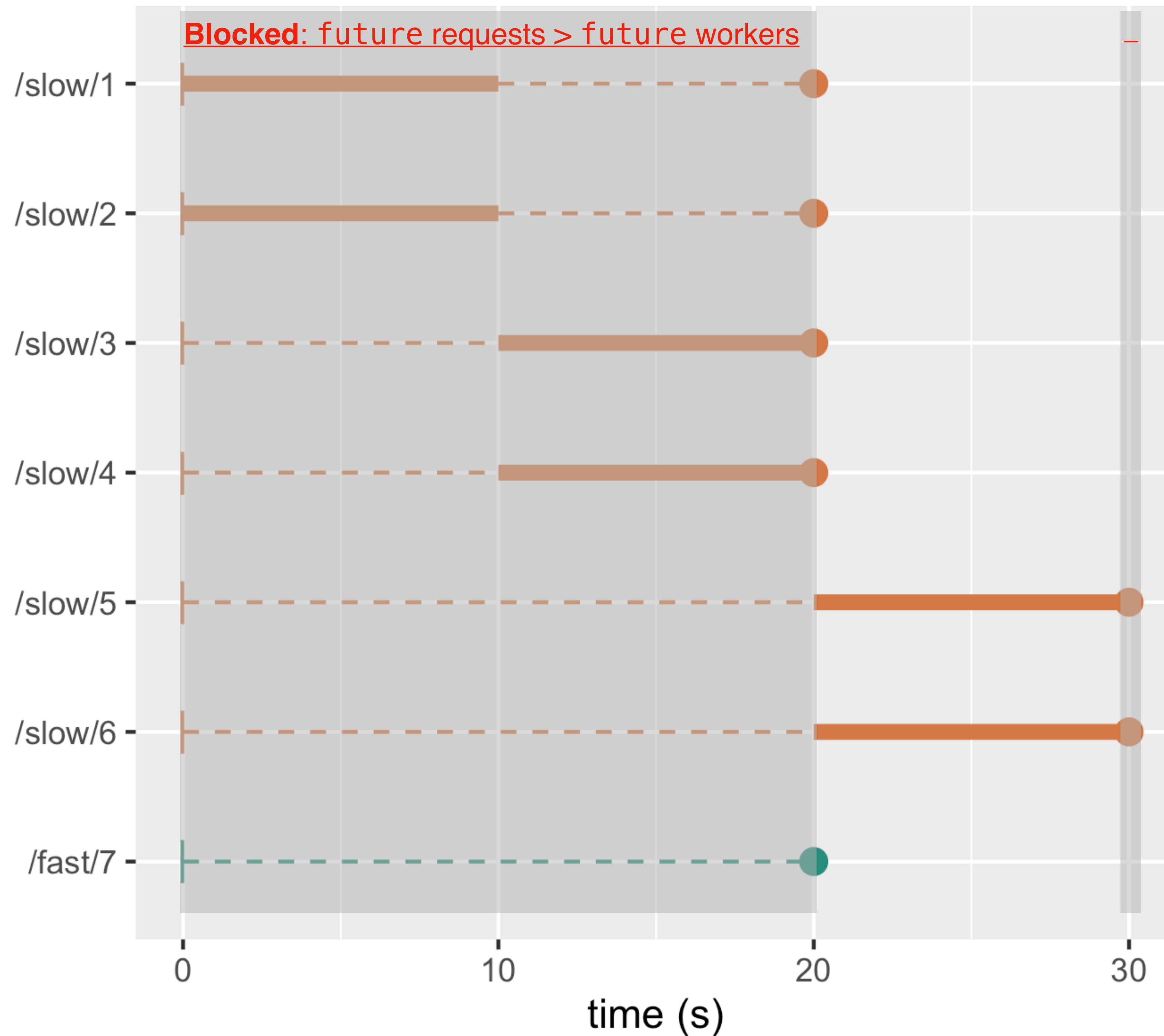
Process



Respond



workers = 2



Route Type

- plumber + future
- plumber

Execution

- Receive
- Respond

Status

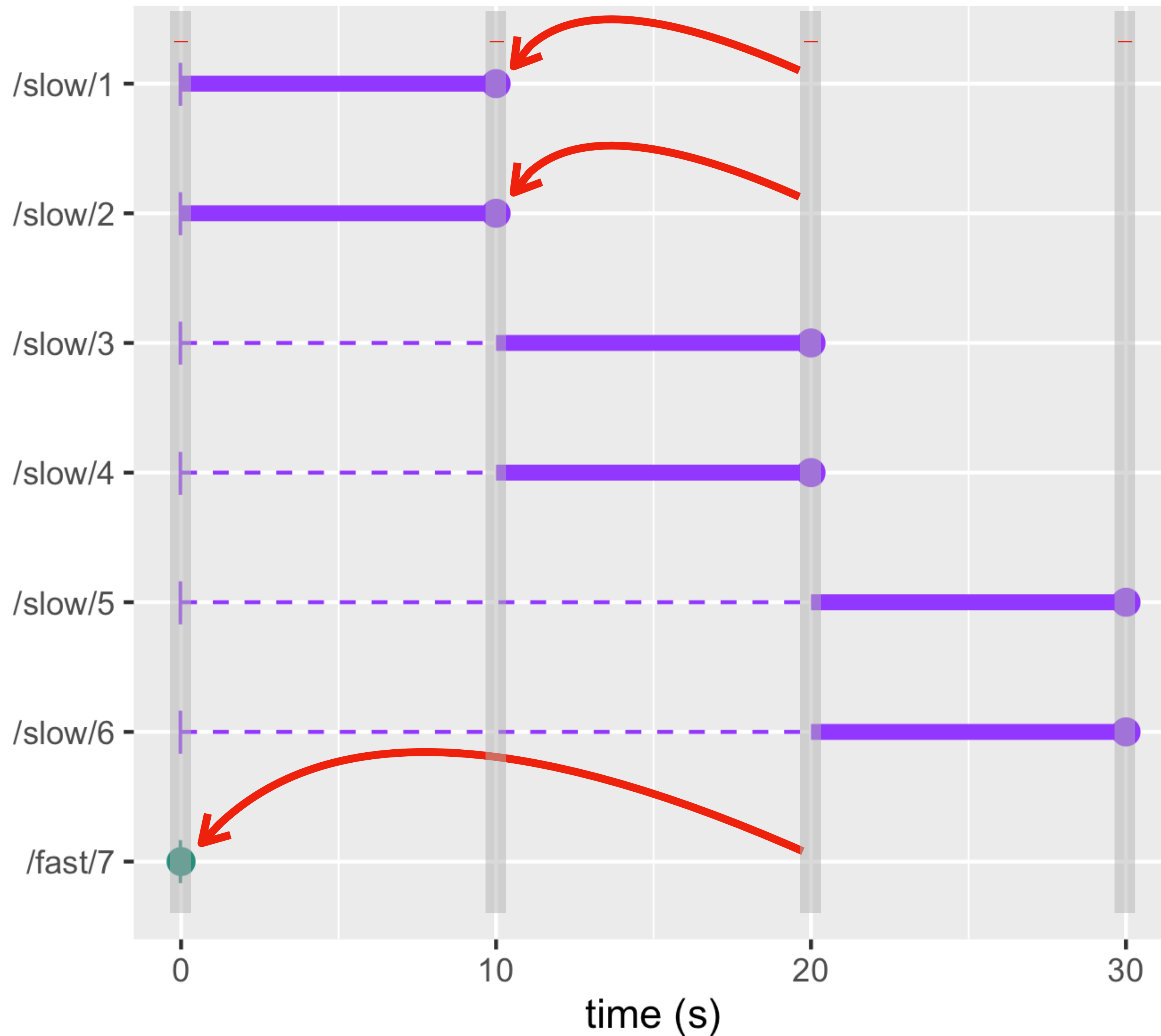
- Waiting
- Working

workers = 2

`promises::future_promise(expr)`

A promise that executes using future

(In final development of
promises v1.2.0)



Route Type

- plumber + future_promise
- plumber

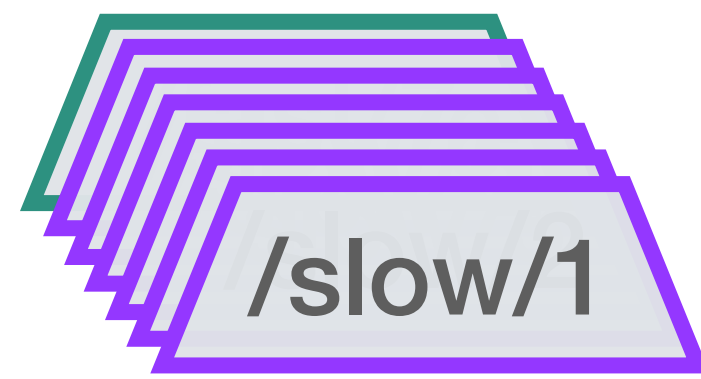
Execution

- Receive
- Respond

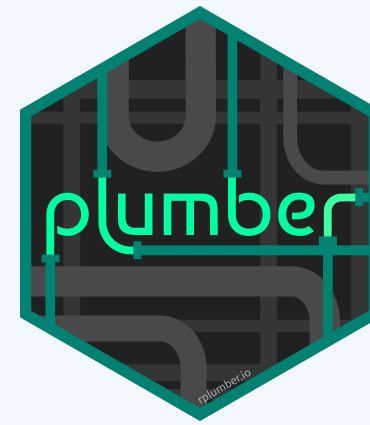
Status

- Waiting in promise
- Working in future

workers = 2



Main



Worker
#1

Worker
#2



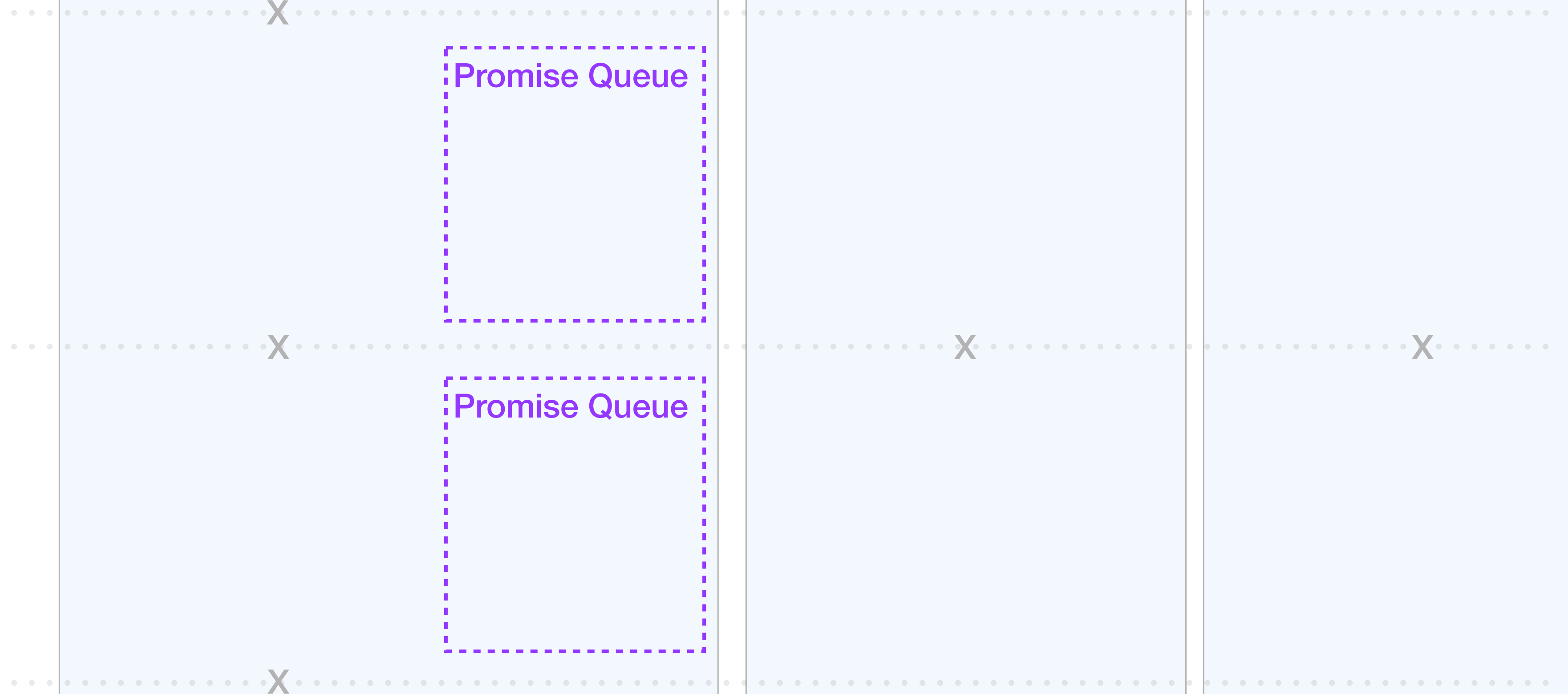
Receive



Process



Respond



workers = 2

